

UNIT 3

1) MVC Architecture

- MVC = Model + View + Controller.
- **Model** → Handles data and database.
- **View** → Displays UI to user.
- **Controller** → Connects Model and View.
- Flow: User → View → Controller → Model → View.
- Advantage: Easy maintenance and organized code.

Example:

- Shopping App:
 - Model = Product data
 - View = Product page
 - Controller = Add to cart logic
-

2) Event Binding & Property Binding in Angular

Property Binding

- Sends data from component → HTML.
- Uses [].

Example:

```
<img [src]="imageUrl">
```

Event Binding

- Handles user actions.
- Sends data from HTML → component.
- Uses ().

Example:

```
<button (click)="save()">Save</button>
```

3) Hooks in ReactJS

Hooks allow functional components to use state and lifecycle features.

Important Hooks

- `useState()` → Manage data/state
- `useEffect()` → Side effects/API calls
- `useContext()` → Share global data
- `useRef()` → Access DOM
- `useMemo()` → Performance optimization

Example:

```
const [count, setCount] = useState(0);
```

4) TypeScript

- TypeScript = Strongly typed JavaScript.
- Developed by Microsoft.
- Adds types, interfaces, OOP support.

Advantages

- Fewer errors
- Better readability
- IDE support
- Scalable

Disadvantages

- Compilation required
- More code
- Learning curve

Example:

```
let name:string = "Kanchan";
```

5) Pipe in Angular

- Pipe transforms data before displaying.

Syntax:

```
{{ value | pipeName }}
```

Built-in Pipes

- uppercase
- date
- currency

Example:

```
{{ name | uppercase }}
```

Output:

KANCHAN

6) Redux Architecture

Redux manages application state.

Main Parts

- Store → Stores data
- Action → What happened
- Reducer → Updates state
- Dispatch → Sends action

Flow:

User → Action → Reducer → Store → UI

Advantage:

- Centralized state management.
-

7) Structural Directives in Angular

Used to change DOM structure.

Types

***ngIf**

Shows/hides element.

```
<p *ngIf="isLogin">Welcome</p>
```

***ngFor**

Repeats elements.

```
<li *ngFor="let item of items">
```

***ngSwitch**

Multiple conditions.

8) TypeScript Modules

Modules organize code into files.

Keywords

- export
- import

Example:

```
export function add(){}
```

```
import {add} from "./math";
```

Advantages:

- Reusability
 - Clean code
 - Easy maintenance
-

9) Features of Web Frameworks

Angular

- Components
- Two-way binding
- Routing

React

- Virtual DOM

- Hooks
- Fast rendering

Django

- Security
 - Admin panel
 - ORM
-

10) (Important Repeated Topic)

Usually based on Angular/React concepts from previous questions.

11) Any 2 Hooks in ReactJS

useState()

Manages state.

```
const [count,setCount]=useState(0);
```

useEffect()

Handles API calls/timers.

```
useEffect(()=>{},[]);
```

12) TypeScript Modules Application

- Create module file.
- Export functions.
- Import into main file.

Example:

```
export function add(){}
```

```
import {add} from "./calculator";
```

13) Pipe in Angular with Code

Built-in Pipe

```
{{ today | date }}
```

Custom Pipe

```
transform(value:string){  
  return value.split("").reverse().join("");  
}
```

14) Angular Application with Multiple Components

Angular app contains many components.

Example:

- Header
- Home
- Footer

Use selector:

```
<app-header></app-header>
```

Advantages:

- Reusable
 - Modular
 - Easy maintenance
-

15) Basic Hooks in ReactJS

useState()

Stores data.

useEffect()

Runs after rendering.

useContext()

Shares global data.

Hooks simplify functional components.

16) Create and Use Component in Angular

Create Component

ng generate component student

Use Component

```
<app-student></app-student>
```

Component contains:

- HTML
- CSS
- TypeScript

17) Functional vs Class Component

Functional Component

- Simple function
- Uses hooks
- No this
- Less code

Example:

```
function Hello(){}
```

Class Component

- Uses class
- Uses lifecycle methods
- Uses this

Example:

```
class Hello extends Component{}
```

Modern React prefers Functional Components.

18) Any 2 Hooks in ReactJS

useState()

Stores dynamic data.

useEffect()

Handles side effects.

Example:

```
useEffect(()=>{  
  console.log("Loaded");  
},[]);
```

19) TypeScript Program for Max & Min Array

Logic

- Compare each element.
- Update max and min.

Example:

```
let arr=[10,5,20];
```

```
let max=arr[0];
```

```
let min=arr[0];
```

Output:

Maximum:20

Minimum:5

20) Types of Directives in Angular

Structural Directive

Changes DOM structure.

Example:

```
*ngIf
```

```
*ngFor
```

Attribute Directive

Changes appearance/style.

Example:

[ngStyle]
[ngClass]

21) Angular Architecture

Main Parts

- Modules
- Components
- Templates
- Services
- Routing
- Directives
- Data Binding

Flow:

User → Template → Component → Service → Server

Advantages:

- Scalable
 - Reusable
 - Modular
-

22) Web Framework

A framework provides ready-made tools for web development.

Reasons to Use

- Faster development
- Security
- Reusability
- Better structure
- Easy maintenance

Example:

Angular, React, Django.

23) AngularJS Features

- JavaScript framework by Google.
- Used for SPA applications.

Features

- Two-way binding
- MVC architecture
- Directives
- Dependency Injection
- Routing
- Filters

Example:

ng-model
ng-repeat

24) Features of Popular Frameworks

Angular

- Components
- Routing
- Two-way binding

React

- Hooks
- Virtual DOM
- Fast rendering

Django

- Security
- ORM
- Admin panel

25) (Usually Practical/Repeated Question)

Mostly related to:

- React hooks
- Angular architecture
- TypeScript modules
- Components/directives

Study repeated concepts carefully.

26) Basic Hooks in ReactJS with Demo

useState()

Counter example.

```
const [count,setCount]=useState(0);
```

useEffect()

Runs after rendering.

```
useEffect( ()=>{  
  alert("Loaded");  
},[]);
```

useContext()

Shares global data.

```
const value = useContext(MyContext);
```

Advantages:

- Easy state management
- Reusable logic
- Cleaner code

UNIT 4

1) ExpressJS as Middleware

- Express.js middleware are functions that run between request and response.
- They can access req, res, and next() function.
- Used for logging, authentication, parsing data, etc.
- next() passes control to next middleware.

Example

```
app.use((req,res,next)=>{  
  console.log("Middleware running");  
  next();  
});
```

Flow

Client → Middleware → Route → Response

2) NoSQL and MongoDB Features

- NoSQL stores unstructured/semi-structured data.
- MongoDB stores data in JSON-like documents.

Features

- Document-oriented database
- Schema-less structure
- High scalability using sharding
- Fast performance with indexing
- Replication for backup
- Easy integration with Node.js

Example

```
{ "name": "Kanchan", "age": 20 }
```

3) Callbacks in Node.js

- Callback = function passed into another function.
- Used for asynchronous programming.
- Prevents blocking execution.

Example

```
function greet(name, callback){  
  console.log(name);  
  callback();  
}
```

Real Life

Food order ready → callback notifies customer.

4) Routes in ExpressJS

- Routes define how app responds to URLs.
- Uses HTTP methods like GET, POST, PUT, DELETE.

Example

```
app.get('/',(req,res)=>{  
  res.send("Home");  
});
```

Flow

Request → Route → Response

5) MongoDB Create, Insert, Delete using Node.js

- Use MongoDB driver to connect database.
- Collection stores documents.

Example

```
collection.insertOne({name:"Kanchan"});  
collection.deleteOne({name:"Kanchan"});
```

Operations

- Create collection
- Insert data

- Delete data
-

6) Node.js Events

- Node.js follows event-driven architecture.
- Uses EventEmitter class.

Example

```
eventEmitter.on('greet',()=>{  
  console.log("Hello");  
});  
eventEmitter.emit('greet');
```

Flow

Event → Listener → Action

7) REST HTTP API in Node.js

- REST API uses HTTP methods for CRUD operations.
- Common methods: GET, POST, PUT, DELETE.

Example

```
app.get('/users',(req,res)=>{  
  res.json(users);  
});
```

Real Life

Food app APIs for orders and menu.

8) CRUD Operations in Node.js

CRUD means:

- Create → POST
- Read → GET
- Update → PUT
- Delete → DELETE

Example

```
app.post('/students',(req,res)=>{});
```

Use

Managing users, students, products.

9) PM2 Microservices

- PM2 is a process manager for Node.js apps.
- Used to manage microservices efficiently.

Features

- Auto restart
- Load balancing
- Monitoring
- Clustering

Command

```
pm2 start app.js
```

10) Template Engine in ExpressJS

- Template engine generates dynamic HTML pages.
- Popular engines: EJS, Pug.

Example

```
app.set('view engine','ejs');
```

EJS

```
<h1><%= name %></h1>
```

11) Advanced MongoDB Features

Features

- Aggregation framework
- Sharding

- Replication
- Transactions
- GridFS
- Indexing
- Change Streams

Use

Large-scale applications.

12) NPM in Node.js

- NPM = Node Package Manager.
- Used to install and manage packages.

Commands

npm install express

npm init -y

Uses

- Dependency management
 - Running scripts
 - Sharing packages
-

13) CRUD in Node.js

- CRUD manages application data.
- Uses Express routes with HTTP methods.

Example

```
app.delete('/items/:id',(req,res)=>{});
```

Use

Student, library, shopping systems.

14) Node.js and File Handling

- Node.js is server-side JavaScript runtime.
- File handling uses fs module.

Operations

- Read → readFile()
- Write → writeFile()
- Append → appendFile()
- Delete → unlink()

Example

```
fs.readFile('data.txt','utf8',callback);
```

15) Routes in ExpressJS (Code)

- Routes map URL to functionality.

Example

```
app.get('/about',(req,res)=>{  
  res.send("About Page");  
});
```

Types

- GET
 - POST
 - PUT
 - DELETE
-

16) Replication in Database

- Replication = copying same data to multiple servers.
- In MongoDB uses Replica Sets.

Advantages

- High availability
- Fault tolerance
- Backup and recovery

- Better performance

Flow

Primary → Secondary servers

17) SQL vs NoSQL

SQL	NoSQL
Table-based	Document-based
Fixed schema	Flexible schema
Vertical scaling	Horizontal scaling
Strong ACID	BASE model

Example

- SQL → Banking
 - NoSQL → Social media
-

18) (Usually skipped in provided list)

19) Event Loop in Node.js

- Event loop handles asynchronous operations.
- Checks Call Stack and Callback Queue.

Flow

Call Stack ↔ Event Loop ↔ Callback Queue

Example

```
setTimeout(()=>{  
  console.log("Done");  
},2000);
```

20) Console Object Methods

Methods

1. `console.log()` → normal output
2. `console.error()` → errors
3. `console.warn()` → warnings
4. `console.table()` → table format

Example

```
console.log("Hello");
```

21) (Question not provided)

22) MapReduce

- Used for processing large datasets.
- Works in Map and Reduce phases.

Steps

1. Map → split data
2. Shuffle → group keys
3. Reduce → final result

Example

Word count:

apple → 3

banana → 2

23) Mongoose ODM

- Library for MongoDB in Node.js.
- Provides schema and model system.

Features

- Schema definition
- Validation
- Query building

- Relationships

Example

```
const Student = mongoose.model('Student',schema);
```

24) CRUD in Node.js

- CRUD = Create, Read, Update, Delete.
- Implemented using REST APIs.

Methods

- POST
- GET
- PUT
- DELETE

Use

Backend data management.

25) (Question not provided)26) (Question not provided)27) (Question not provided)

28) API and REST HTTP APIs

- API allows communication between applications.
- REST API uses HTTP methods.

Features

- Stateless
- Resource-based
- JSON data format

Example

```
app.get('/students',(req,res)=>{});
```

29) MongoDB Create, Display & Search Data

Operations

- Create collection
- Insert records
- Display data
- Search records

Example

```
students.findOne({name:"Rahul"});
```

30) File System Module in Node.js

- fs module manages files.

Methods

- appendFile()
- unlink()

Example

```
fs.appendFile('demo.txt',"Hello",callback);
```

31) Socket Programming in Node.js

- Socket enables real-time communication.
- Uses net module.

Server

```
const server = net.createServer();
```

Client

```
const client = net.createConnection({port:5000});
```

Uses

- Chat applications
- Live notifications
- Gaming systems

UNIT 5

1) Mobile-First and Mobile Web

Mobile-First

- Mobile-First is a web design approach where website design starts for mobile devices first.
- After mobile design, it is expanded for tablets and desktops.
- Focuses on simple UI, fast loading, and essential content.

Mobile Web

- Mobile Web means websites accessed through mobile browsers.
- These websites are optimized for small screens and touch usage.
- Users can access content anytime using internet/Wi-Fi.

Difference

Feature	Mobile-First	Mobile Web
Meaning	Design strategy	Mobile website access
Focus	Mobile UI first	Mobile browsing
Purpose	Better design	Better accessibility

Mobile Devices

1. Smartphones
2. Tablets
3. Smartwatches
4. Feature Phones
5. PDAs
6. Phablets

Conclusion

- Mobile-First improves mobile user experience.
- Mobile Web allows easy access to websites on mobile devices.

2) jQuery Mobile and Framework

jQuery Mobile

- jQuery Mobile is a touch-friendly framework for creating mobile web applications.
- Built using jQuery, HTML5, CSS, and JavaScript.
- Provides ready-made UI components.

Features

1. Touch Support
2. Responsive Design
3. Cross-Platform Support
4. AJAX Navigation
5. Themes and Styling

Simple Structure

```
<div data-role="page">  
  
  <div data-role="header">Header</div>  
  
  <div data-role="content">Content</div>  
  
  <div data-role="footer">Footer</div>  
  
</div>
```

Conclusion

- jQuery Mobile makes mobile app development simple and fast.

3) Any 3 Events in jQuery Mobile

1) pagecreate

- Triggered when page is created first time.

```
$(document).on("pagecreate", function(){  
  
  alert("Page Created");  
  
});
```

2) pageshow

- Triggered when page becomes visible.

```
$(document).on("pageshow", function(){  
    alert("Page Visible");  
});
```

3) pagehide

- Triggered when page is hidden.

```
$(document).on("pagehide", function(){  
    alert("Page Hidden");  
});
```

Conclusion

- Events make applications interactive and dynamic.
-

4) jQuery vs jQuery Mobile

Feature	jQuery	jQuery Mobile
Type	JavaScript Library	Mobile Framework
Purpose	Web Development	Mobile Apps
UI Components	No	Yes
Touch Support	Limited	Full
Responsiveness	Manual	Built-in

Conclusion

- jQuery is for general web scripting.
 - jQuery Mobile is for mobile-friendly UI development.
-

5) Advantages and Disadvantages of jQuery Mobile

Advantages

1. Easy Development

2. Responsive Design
3. Cross-Platform Support
4. Touch-Friendly UI
5. Ready Themes

Disadvantages

1. Slow Performance
2. Outdated Technology
3. Limited Customization
4. Depends on jQuery
5. Not good for large apps

Conclusion

- Good for simple mobile apps but less useful for modern large projects.
-

6) Header and Footer Code

```
<div data-role="page">
```

```
  <div data-role="header">
```

```
    <h1>Header</h1>
```

```
  </div>
```

```
  <div data-role="content">
```

```
    <p>Main Content</p>
```

```
  </div>
```

```
  <div data-role="footer">
```

```
    <h4>Footer</h4>
```

```
  </div>
```

</div>

Conclusion

- Header is top section and footer is bottom section of page.

7) Mobile Devices and Desktop Devices

Mobile Devices

- Portable devices with touch interface.
- Example: Smartphones, Tablets, Smartwatches.

Features

- Portable
- Battery Powered
- Wireless Connectivity

Desktop Devices

- Fixed computers used at homes/offices.
- Example: Desktop PCs, Workstations.

Features

- High Performance
- Large Storage
- Keyboard and Mouse

Difference

Mobile	Desktop
Portable	Fixed
Touchscreen	Keyboard/Mouse
Moderate Power	High Power

8) Mobile-First Design

Meaning

- Website design starts for mobile screens first.
- Later expanded for larger screens.

Benefits

1. Faster Loading
2. Better User Experience
3. Better SEO
4. Responsive Design
5. Easy Navigation

Conclusion

- Mobile-First is important because most users access websites using mobiles.
-

9) Navigation in jQuery Mobile

Meaning

- Navigation means moving from one page to another.

Example

```
<a href="about.html" data-role="button">
```

Go to About

```
</a>
```

Features

- Smooth transitions
 - AJAX-based navigation
 - Better user experience
-

10) Layouts in jQuery Mobile

Types

1) Single Page Layout

- One page structure.

2) Multi-Page Layout

- Multiple pages in one file.

3) Grid Layout

- Row and column arrangement.

4) Panel Layout

- Sliding side menu.

Conclusion

- Layouts help organize mobile UI properly.
-

11) Importance of CSS in Mobile Websites

Importance

1. Responsive Design
2. Better UI
3. Touch-Friendly Design
4. Faster Loading
5. Consistent Layout

Example

```
.mobile-btn{  
width:100%;  
padding:10px;  
}
```

Conclusion

- CSS improves appearance and usability of mobile websites.
-

12) Any Five Widgets

Five Widgets

1. Button
2. Listview
3. Navbar

4. Slider

5. Dialog

Two Widgets

Button Widget

```
<a href="#" data-role="button">Click</a>
```

- Used for actions and navigation.

Listview Widget

```
<ul data-role="listview">
```

```
<li>Item 1</li>
```

```
</ul>
```

- Displays data in list format.
-

13) Any 3 Widgets

1) Button Widget

- Creates clickable buttons.

2) Listview Widget

- Displays items in list format.

3) Navbar Widget

```
<div data-role="navbar">
```

```
</div>
```

- Creates navigation menu.

Conclusion

- Widgets make UI interactive and mobile-friendly.
-

14) Concept of Role in jQuery Mobile

Meaning

- data-role defines the function of an element.

Important Roles

1) data-role="page"

- Main page container.

2) data-role="header"

- Top section of page.

3) data-role="content"

- Main content area.

4) data-role="footer"

- Bottom section.

Conclusion

- Roles help create structured mobile pages easily.
-

15) Hide Button on Click

```
<script>
$(document).on("pagecreate", function(){
    $("#btn").click(function(){
        $(this).hide();
    });
});
</script>
```

```
<a href="#" id="btn" data-role="button">
```

Hide Me

```
</a>
```

Conclusion

- hide() method hides button after clicking.
-

16) Content in jQuery Mobile

Meaning

- Content is the main display area of a page.

Code

```
<div data-role="content">  
  
<p>Main Content Area</p>  
  
</div>
```

Uses

- Displays text, images, forms, buttons.
-

17) jQuery and its Architecture

jQuery

- Lightweight JavaScript library.
- Simplifies web development.

Architecture Layers

1. Core Layer
2. Selector Engine
3. DOM Manipulation
4. Event Handling
5. AJAX Layer
6. Effects Layer

Example

```
$("#btn").click(function(){  
  
    $("#text").hide();  
  
});
```

Conclusion

- jQuery architecture is modular and easy to use.
-

18) CDN in Web Development

CDN Meaning

- Content Delivery Network.
- Network of servers delivering content faster.

Advantages

1. Faster Speed
2. Better Performance
3. Reduced Server Load
4. Security
5. Scalability

Example

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

Conclusion

- CDN improves website speed and reliability.
-

19) Is jQuery a Mobile Framework?

Answer

- No, jQuery itself is not a mobile framework.
- jQuery Mobile is the mobile framework built on jQuery.

Difference

jQuery	jQuery Mobile
JS Library	Mobile Framework
General Web	Mobile Apps

Conclusion

- jQuery supports frameworks like jQuery Mobile.
-

20) Page in jQuery Mobile

Meaning

- A page is the main container of mobile screen.

Code

```
<div data-role="page">
```

```
<div data-role="header">
```

```
<h1>Header</h1>
```

```
</div>
```

```
<div data-role="content">
```

```
Content Area
```

```
</div>
```

```
<div data-role="footer">
```

```
Footer
```

```
</div>
```

```
</div>
```

Conclusion

- Page organizes mobile app structure.

21) Content in jQuery Mobile

Meaning

- Content section contains actual page information.

Code

```
<div data-role="content">
```

```
<p>Welcome User</p>
```

```
</div>
```

Conclusion

- Content area is used for user interaction and display.

22) Page in jQuery Mobile

Features

1. Main container
2. Responsive
3. Supports header/content/footer
4. Multi-page support

Code

```
<div data-role="page">  
  
</div>
```

Conclusion

- Page is the foundation of jQuery Mobile apps.
-

23) Slide Up Transition in jQuery Mobile

Meaning

- Slideup transition gives upward animation during navigation.

Code

```
<a href="#about"  
data-transition="slideup">  
Go  
</a>
```

Advantages

1. Smooth Navigation
2. Better UI
3. Mobile App Feel

Conclusion

- Slide transitions improve user experience.

24) What is Page? Write code to create a page in jQuery Mobile

Meaning

- A page in jQuery Mobile is the main container of a mobile screen.
- It is created using data-role="page".
- A page usually contains:
 1. Header
 2. Content
 3. Footer

Features

1. Main structure of mobile app
2. Supports responsive design
3. Can contain multiple pages in one file
4. Easy navigation between pages

Code

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet"
href="https://code.jquery.com/mobile/1.4.5/jquery.mobile-1.4.5.min.css">

<script src="https://code.jquery.com/jquery-1.11.1.min.js"></script>

<script src="https://code.jquery.com/mobile/1.4.5/jquery.mobile-
1.4.5.min.js"></script>
</head>

<body>

<div data-role="page">

<div data-role="header">
<h1>My Page</h1>
</div>
```

```
<div data-role="content">
  <p>Welcome to jQuery Mobile</p>
</div>
```

```
<div data-role="footer">
  <h4>Footer</h4>
</div>
```

```
</div>
```

```
</body>
</html>
```

Conclusion

- A page is the basic building block of jQuery Mobile applications.
-

25) Slide Up Transition in jQuery Mobile

Meaning

- Slide Up Transition is an animation effect used while moving from one page to another.
- The next page appears by sliding upward.
- It creates a smooth mobile-app-like effect.

Features

1. Smooth page navigation
2. Better user experience
3. Mobile-friendly animation
4. Uses data-transition="slideup"

Code

```
<!DOCTYPE html>
<html>
<head>

<link rel="stylesheet"
href="https://code.jquery.com/mobile/1.4.5/jquery.mobile-1.4.5.min.css">
```

```
<script src="https://code.jquery.com/jquery-1.11.1.min.js"></script>
```

```
<script src="https://code.jquery.com/mobile/1.4.5/jquery.mobile-1.4.5.min.js"></script>
```

```
</head>
```

```
<body>
```

```
<!-- Home Page -->
```

```
<div data-role="page" id="home">
```

```
  <div data-role="header">
```

```
    <h1>Home Page</h1>
```

```
  </div>
```

```
  <div data-role="content">
```

```
    <a href="#about"
```

```
      data-role="button"
```

```
      data-transition="slideup">
```

```
      Go to About
```

```
    </a>
```

```
  </div>
```

```
</div>
```

```
<!-- About Page -->
```

```
<div data-role="page" id="about">
```

```
  <div data-role="header">
```

```
    <h1>About Page</h1>
```

```
  </div>
```

```
  <div data-role="content">
```

```
    <p>Welcome to About Page</p>
```

```
  </div>
```

```
</div>
```

```
</body>
```

```
</html>
```

Conclusion

- Slide Up Transition improves navigation and makes applications more interactive.

UNIT 6

1) Types of Elastic Load Balancer (ELB)

Definition

- ELB distributes incoming traffic across multiple EC2 servers.
- Improves scalability, availability, and fault tolerance.

Types of ELB

1) Application Load Balancer (ALB)

- Works at Layer 7 (Application Layer).
- Handles HTTP/HTTPS traffic.
- Supports URL and host-based routing.
- Best for web applications and microservices.

2) Network Load Balancer (NLB)

- Works at Layer 4 (Transport Layer).
- Handles TCP, UDP, TLS traffic.
- Very high speed and low latency.
- Best for real-time applications.

3) Classic Load Balancer (CLB)

- Older AWS load balancer.
- Works at Layer 4 and Layer 7.
- Used for simple and legacy applications.

4) Gateway Load Balancer (GWLB)

- Used for security appliances like firewalls.
- Performs traffic inspection and balancing.
- Helps secure network traffic.

Diagram

Users → ELB → Multiple EC2 Instances

Conclusion

- Different ELB types are used for different networking needs.
- ELB improves performance and reliability.

2) Components of VPC

Definition

- VPC is a private virtual network in AWS.
- Provides secure and isolated cloud environment.

Components

1) Subnets

- Divide VPC into smaller networks.
- Public subnet → internet accessible.
- Private subnet → secure internal network.

2) Internet Gateway (IGW)

- Connects VPC to internet.

3) Route Table

- Defines path for network traffic.

4) NAT Gateway

- Allows private subnet to access internet securely.

5) Security Groups

- Firewall at instance level.

6) Network ACL

- Firewall at subnet level.

7) Elastic IP

- Static public IP address.

8) VPC Peering

- Connects two VPCs privately.

Diagram

Internet → IGW → VPC → Public/Private Subnets

Conclusion

- VPC components provide security, routing, and connectivity.

3) Elastic Beanstalk and Advantages

Definition

- Elastic Beanstalk is AWS PaaS service.
- Used to deploy and manage applications easily.

Working

- Developer uploads code.
- AWS creates EC2, Load Balancer, Auto Scaling automatically.

Advantages

1. Easy deployment
2. Automatic scaling
3. Cost effective
4. Monitoring support
5. High availability
6. Multi-language support

Diagram

Code → Elastic Beanstalk → EC2 + Load Balancer

Conclusion

- Developers focus on coding instead of infrastructure management.
-

4) Any Three AWS Storage Services

Definition

- AWS provides cloud storage services for different needs.

1) Amazon S3

- Object storage service.
- Stores files in buckets.
- Used for images, videos, backups.

2) Amazon EBS

- Block storage for EC2.
- Works like hard disk.
- Used for databases.

3) Amazon EFS

- Shared file storage.
- Accessible by multiple EC2 instances.

Diagram

S3 → Object Storage

EBS → Block Storage

EFS → File Storage

Conclusion

- AWS offers scalable and secure storage services.
-

5) Elastic Load Balancer and Working

Definition

- ELB distributes traffic among multiple servers.

Working

1. Receives user request
2. Checks server health
3. Distributes traffic evenly
4. Sends response back to user

Diagram

Users → ELB → Server1/Server2/Server3

Conclusion

- ELB improves application availability and performance.
-

6) AWS Cloud and Services

Definition

- AWS is a cloud platform providing on-demand services.

Services

1. Compute → EC2
2. Storage → S3, EBS
3. Database → RDS

4. Networking → VPC
5. Security → IAM
6. Monitoring → CloudWatch

Conclusion

- AWS provides scalable and flexible cloud services.
-

7) Features and Benefits of AWS Cloud

Features

1. Scalability
2. Security
3. Flexibility
4. Global infrastructure
5. Pay-as-you-go

Benefits

1. Cost saving
2. High performance
3. Reliability
4. Easy deployment
5. Business growth support

Conclusion

- AWS provides secure and reliable cloud environment.
-

8) S3 Bucket and Creation Steps

Definition

- S3 Bucket stores files in AWS cloud.

Steps to Create Bucket

1. Login AWS Console
2. Open S3 service

3. Click Create Bucket
4. Enter bucket name
5. Select region
6. Configure settings
7. Create bucket

Conclusion

- S3 bucket provides durable and scalable storage.
-

9) Components of VPC

Components

1. Subnets
2. Internet Gateway
3. Route Table
4. NAT Gateway
5. Security Groups
6. Network ACL
7. Elastic IP
8. VPC Peering
9. DHCP Options Set

Conclusion

- VPC components help manage secure cloud networking.
-

10) PuTTY and EC2 Connection

Definition

- PuTTY is SSH client used for remote server access.

Steps

1. Launch EC2 instance
2. Convert .pem to .ppk using PuTTYgen

3. Open PuTTY
4. Enter public IP
5. Add .ppk key
6. Click Open

Conclusion

- PuTTY helps securely manage EC2 instances.
-

11) Deploy Website/Application on AWS

Steps

1. Launch EC2 instance
2. Connect to server
3. Install web server
4. Upload website files
5. Configure security group
6. Access website using public IP

Conclusion

- AWS makes deployment fast and scalable.
-

12) EC2 Service and Website Deployment

Definition

- EC2 provides virtual servers in AWS cloud.

Steps

1. Launch EC2
2. Configure security group
3. Connect using SSH/PuTTY
4. Install Apache/Nginx
5. Upload files
6. Access via browser

Conclusion

- EC2 is flexible for website hosting.
-

13) AWS Cloud and Two Services

AWS Cloud

- Provides cloud services over internet.

Services

1) EC2

- Virtual server service.

2) S3

- Object storage service.

Conclusion

- AWS services support scalable applications.
-

14) ELB, Types and Advantages

Types

1. ALB
2. NLB
3. CLB
4. GWLB

Advantages

- High availability
- Scalability
- Fault tolerance
- Better performance
- Security

Conclusion

- ELB ensures balanced traffic distribution.
-

15) VPC and Components

Definition

- VPC is isolated virtual network in AWS.

Components

- Subnets
- IGW
- Route Table
- NAT Gateway
- Security Groups
- Network ACL

Conclusion

- VPC improves cloud security and networking.
-

17) Advantages of VPC

Advantages

1. Network isolation
2. Better security
3. Flexible IP addressing
4. Public/private subnet support
5. Controlled internet access
6. Easy connectivity
7. High availability
8. Scalability

Conclusion

- VPC provides secure and flexible networking.
-

18) AWS Storage Types

Types

1. Object Storage → S3

2. Block Storage → EBS
3. File Storage → EFS
4. Archive Storage → Glacier
5. Instance Storage

Conclusion

- AWS provides different storage solutions for different requirements.
-

19) Cloud Computing and Benefits

Definition

- Cloud computing provides IT services over internet.

Benefits

1. Cost saving
2. Scalability
3. High availability
4. Flexibility
5. Backup and recovery
6. Security
7. Easy deployment

Conclusion

- Cloud computing is efficient and cost-effective.
-

20) Elastic Beanstalk and Working

Working

1. Upload application code
2. AWS creates environment
3. Configures EC2 and web server
4. Load balancing and auto scaling
5. Monitoring using CloudWatch

Conclusion

- Elastic Beanstalk automates deployment and scaling.
-

21) VPC and Components

Components

- Subnets
- IGW
- Route Tables
- NAT Gateway
- Security Groups
- Network ACL
- Elastic IP
- VPC Peering

Conclusion

- VPC creates secure AWS network environment.
-

22) Cloud Computing and Benefits

Benefits

- Cost efficiency
- Scalability
- Accessibility
- Security
- Backup and recovery

Conclusion

- Cloud computing provides flexible IT resources.
-

23) Steps to Deploy Application on Elastic Beanstalk

Steps

1. Prepare application
2. Login AWS
3. Create application

4. Create environment
5. Upload code
6. Configure settings
7. Launch environment
8. Monitor deployment
9. Access application URL

Conclusion

- Elastic Beanstalk simplifies application deployment.
-

24) AWS Storage Services

Services

1. Amazon S3 → Object storage
2. Amazon EBS → Block storage
3. Amazon EFS → File storage
4. Amazon Glacier → Archive storage
5. AWS Backup → Backup service
6. Storage Gateway → Hybrid storage

Conclusion

- AWS storage services provide secure and scalable storage solutions.
-

25) PuTTY and EC2 Connection

Steps

1. Launch EC2
2. Convert .pem to .ppk
3. Open PuTTY
4. Enter host name
5. Select .ppk key
6. Connect to instance

Conclusion

- PuTTY provides secure remote EC2 access.
-

26) Cloud Computing, Benefits and Types

Types

1. Public Cloud
2. Private Cloud
3. Hybrid Cloud

Benefits

- Cost saving
- Scalability
- Security
- Flexibility

Conclusion

- Cloud computing offers scalable and reliable services.
-

27) Steps to Deploy Application on Elastic Beanstalk

Steps

1. Prepare application
2. Login AWS Console
3. Create application
4. Create environment
5. Upload code
6. Configure settings
7. Deploy application
8. Monitor application
9. Access URL

Conclusion

- Elastic Beanstalk automates deployment process.

28) Steps to Deploy Website on EC2

Steps

1. Launch EC2 instance
2. Configure security group
3. Connect to EC2
4. Install Apache/Nginx
5. Upload website files
6. Start web server
7. Access website

Conclusion

- EC2 provides scalable website hosting.
-

29) Connect EC2 Instance with PuTTY

Steps

1. Open PuTTYgen
2. Convert .pem to .ppk
3. Open PuTTY
4. Enter public IP
5. Add .ppk key
6. Connect to EC2

Conclusion

- PuTTY allows secure SSH connection to EC2 servers.